JURIS VAULT AI

System Architecture & Architecture Decision Records

The complete engineering and infrastructure reference for Juris Vault AI - prepared for the infrastructure team and the AWS platform team.

Owner: Jishan - Juris Vault AI Team

Document version: 1.4 - 23 October 2026

System version: 0.1.0 (Cargo workspace)

Contents: Part I - System Architecture | Part II - ADR-0001 to ADR-0018

Status labels used throughout: **LIVE-VERIFIED** = proven end to end in production-like runs. **BUILT** = implemented, verification partial. **TARGET** = design for the AWS move, not yet deployed. **PROPOSED** = fully designed, awaiting the owner's decision - do not treat as approved spend. Companion files in the repository: README.md, CHANGELOG.md, docs/ARCHITECTURE.md, docs/ADR.md, DEPLOY.md.

CONTENTS

What is in this document

Part I - System Architecture. What the system is, how it is built, how data flows through it, how it is secured, how it is deployed today, and the AWS target topology in full (Section 12).

1. What the system is

2. System context

3. Container & component architecture

4. Backend architecture (Rust workspace)

5. Retrieval & generation pipeline

6. Frontend architecture

7. Data architecture

8. Security architecture

9. Billing architecture

10. Performance & measured capacity (21 Oct 2026)

11. Reliability & failure modes

12. Deployment architecture

13. Observability

14. Testing & benchmarks

15. Limitations & roadmap (summary)

Appendix A — Configuration reference

Appendix B — API inventory (65 routes, `/api/v1`)

Appendix C — Port & process inventory

Part II - Architecture Decision Records. Every significant decision, with its context, the alternatives that were rejected, and the measured evidence behind the choice.

ADR-0001 — Rust monorepo: five-crate workspace, single-binary server

ADR-0002 — RAG architecture: faithful RAG-Anything port + LightRAG modes

ADR-0003 — Embeddings: bge-m3 fixed; improve via a decision layer

ADR-0004 — Reranker: bge-reranker-v2-m3 local sidecar as decision layer

ADR-0005 — Hybrid retrieval: vector + BM25 with RRF fusion

ADR-0006 — Vector store: in-memory default, SereneDB persistent backend

ADR-0007 — OCR: Surya-only pipeline

ADR-0008 — LLM provider: Cloudflare Workers AI today; provider abstracted

ADR-0009 — Parent-child chunking for match/context separation

ADR-0010 — Persistence: atomic JSON stores now; PostgreSQL migration path

ADR-0011 — Auth: Argon2 + JWT, role-based admin, optional OIDC

ADR-0012 — Confidential Vault: Argon2id → AES-256-GCM, memory-only key

ADR-0013 — Billing: token metering; mock + YouPI-PG payments

ADR-0014 — Web research: self-hosted sidecars, primary-source-first, citation audit

ADR-0015 — Deployment today: single host / Docker; sandbox relay is dev-only

ADR-0016 — AWS target architecture (PROPOSED)

ADR-0017 — Data residency & compliance posture: India-first

ADR-0018 — Adopt functionality, never identity, from reference products

ADR-0019 — Fusion suite, rank-interleave leg merging, retrieval cache

ADR-0020 — Vector engines: Spice's lineup behind the VectorStore trait

The open decisions register at the end of Part II lists the decisions the infra/AWS programme is waiting on, and who owns each.

PART I

System Architecture

Document	System Architecture (engineering, infrastructure & AWS platform teams)
Product	Juris Vault AI — legal document intelligence for Indian law practice
Owner	**Jishan** — Juris Vault AI Team
Version	1.6 — 23 October 2026
System version	0.1.0 (Cargo workspace version)
Companion documents	ADR.md (decision records ADR-0001…0018) · README.md · CHANGELOG.md · DEPLOY.md
Status labels	**LIVE-VERIFIED** = proven end-to-end in production-like runs · **BUILT** = implemented, verification partial · **TARGET** = design for the AWS move, not yet deployed · **ROADMAP** = agreed direction, not built

> Read this first if you are joining the infra/AWS effort. Section 12 is the part that needs you: today's runtime is a development sandbox behind a relay; the AWS target topology is fully specified there and in ADR-0016.

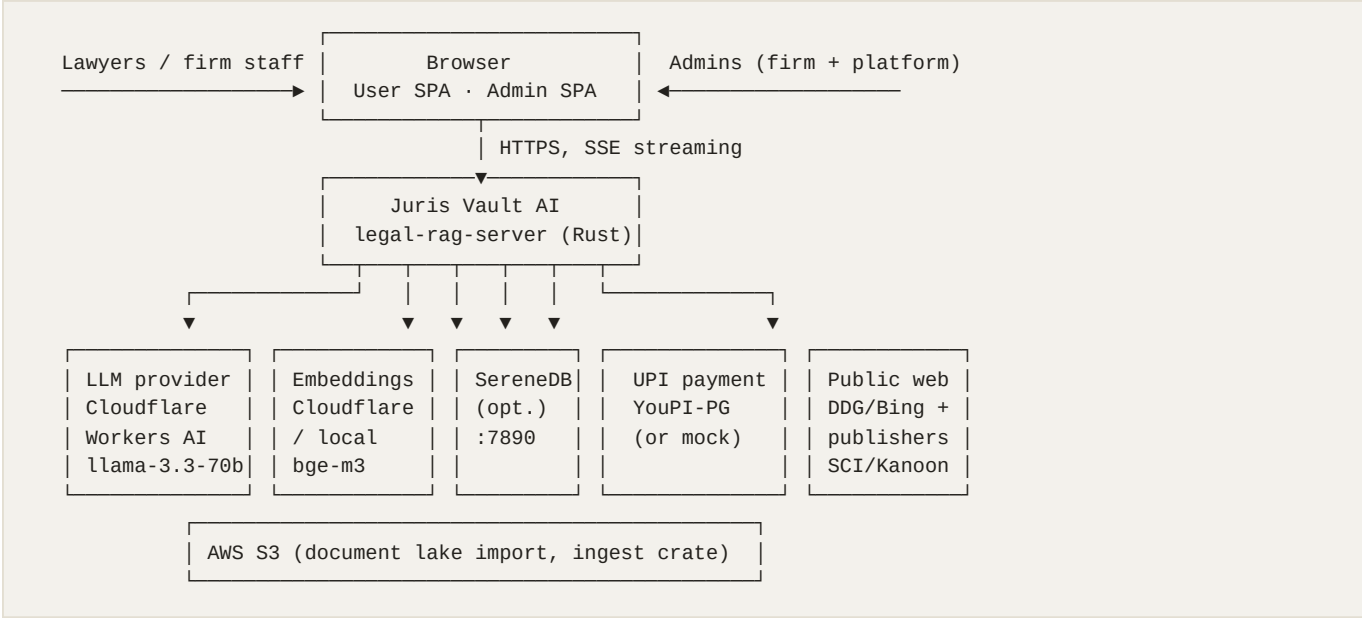
1. What the system is

Juris Vault AI is a retrieval-augmented generation (RAG) platform for law firms: users upload legal documents (statutes, judgments, contracts, scanned court papers — English and Hindi), and the system answers questions, drafts documents, runs structured research, and executes multi-step legal workflows over that corpus. Every generated statement is grounded in retrieved sources, cited at chunk level, and — for web-blended answers — passed through a citation-integrity audit that strips unsupported citations before the user sees them.

Design principles, in priority order:

1. **Provenance over fluency.** A citation the model cannot support is worse than no citation. The integrity audit, primary-source ranking, and source viewer all exist to serve this.
2. **Confidentiality is architectural, not cosmetic.** Brief-scoped retrieval and the encrypted Private Vault enforce boundaries in the retrieval layer, not in the UI.
3. **India-first.** Indic OCR (Surya), Hindi↔English cross-lingual retrieval, Indian primary authorities (Supreme Court, Indian Kanoon, India Code), INR pricing, data residency target `ap-south-1` (Mumbai).
4. **Measure, then decide.** Retrieval changes ship only with offline bench numbers; model changes ship only after head-to-heads (see §14).
5. **Honest surfaces.** Roadmap features are labelled roadmap in-product; verified vs unverified status is tracked in the changelog.

2. System context



External dependencies and their failure behaviour:

Dependency	Used for	If it fails
LLM endpoint (OpenAI-compatible; today Cloudflare Workers AI)	Answer generation, graph extraction, research, workflows, interact/draft	Query path retries with backoff (4 attempts), then a friendly error; cached answers keep serving
Embeddings endpoint (today Cloudflare; local sidecar exists)	Ingest + query vectors	Same retry policy; cache hits unaffected
SereneDB (optional)	Persistent vector store	Falls back to in-memory index with a warning at boot
YouPI-PG (or mock)	UPI checkout	Checkout unavailable; metering/quota unaffected
Public web (DDG/Bing HTML + stealth browser fallback)	Web blending, research evidence	Web leg returns vault-only answers; per-source failures tolerated
AWS S3	Bulk document import	Import endpoint errors; uploads via UI unaffected

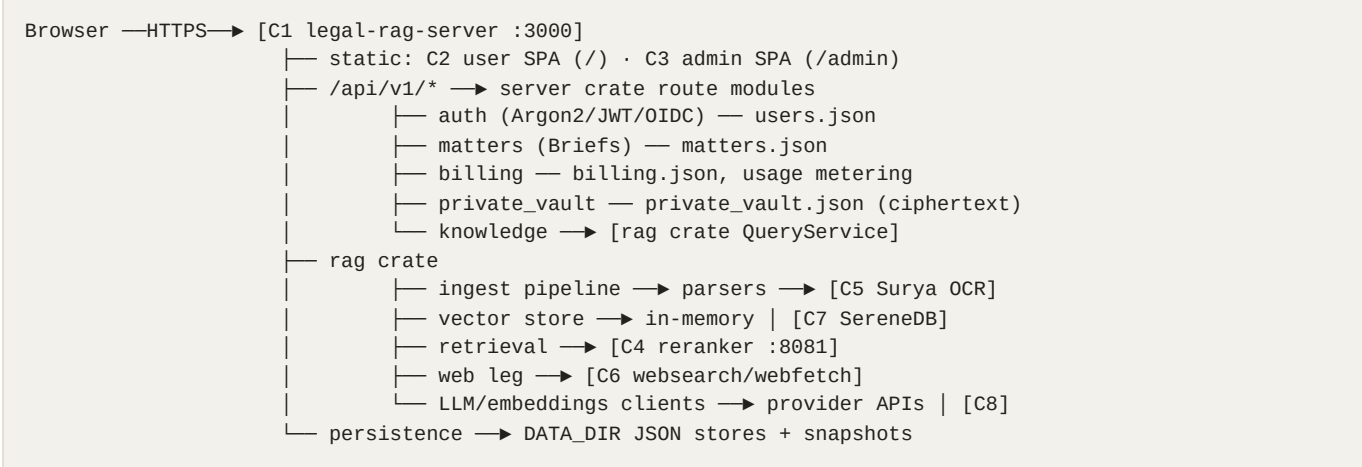
3. Container & component architecture

3.1 Runtime components

#	Component	Tech	Port	Role
C1	`legal-rag-server`	Rust (Axum 0.8 / Tokio 1), single static-ish binary	**3000**	API (65 routes), serves both SPAs, all product logic
C2	User SPA	React 19.2 · TS 6 · Vite 8 · Tailwind 3.4	served by C1 at `/`	Ask, Vault, Briefs, Interact, Draft, Research, Agents, Private Vault, Plans & Billing, Dashboard
C3	Admin SPA ("Command Center")	React 19.2 · separate Vite app	served by C1 at `/admin`	Users, corpus, settings/prompts, billing, insights, golden rules, Verified Answers, hit-test
C4	Reranker sidecar	Python (sentence-transformers), bge-reranker-v2-m3	**8081**	Cross-encoder scoring of the fused retrieval pool

#	Component	Tech	Port	Role
C5	OCR sidecar	Python, Surya 0.22 (+ llama.cpp backend)	subprocess	OCR for scanned PDF pages (text layer first; OCR only where needed)
C6	Web sidecars	Python (`tools/web/websearch.py` , `webfetch.py`) + stealth Firefox fallback	subprocess	Search, primary-source probes, page/PDF fetching and extraction
C7	SereneDB (optional)	`serenedb/serenedb` image	**7890**	Vector store over Postgres wire protocol (`FLOAT[N]` , `<=>` , IVF index)
C8	Embeddings sidecar (test/local)	Python, bge-m3 fp16, OpenAI-compatible	**8083**	Local embeddings — removes provider dependency; test stack uses it by default
C9	Gemini bridge (TEST ONLY)	gemini-web2api, anonymous mode	**8082**	Lets the isolated test instance use Gemini Flash as its LLM; **never** in the production path
C10	Relay agent (DEV/DEMO ONLY)	Node (`tools/tunnel/vm-agent.js`) + Cloudflare Worker	—	Exposes the sandbox build at a public URL. **Not a production component** — replaced by ALB in the AWS target

3.2 Component diagram



4. Backend architecture (Rust workspace)

Cargo workspace, resolver 2, edition 2021, version 0.1.0. Five crates with a strict dependency direction (`server → rag/ingest/auth → common`):

Crate	Contents (key modules)
`common`	Env-driven `Config` , shared `Error` type
`auth`	Argon2 password hashing, JWT access/refresh issuance + `AuthUser` / `AdminUser` Axum extractors, Auth0-compatible OIDC client (login/callback)
`ingest`	S3 source: list/download raw object bytes for the pipeline (`LEGAL_RAG_S3_*`)
`rag`	The intelligence core — see §5. Modules: `config` , content blocks + `parsers` , `ContextExtractor` , modal processors, `pipeline` (incl. parent-child split), `graph` extraction, `bm25` , `query` (six modes, caches, rerank), `audit` (citation integrity), `web` (StealthWebSearch), `research` , `agent` (workflows), `cache` , `persist` , `usage` (metering), vector stores (in-memory, `SereneDbVectorStore`)
`server`	Axum wiring + route modules: auth, vault, knowledge, matters, billing, private vault, settings, dashboard, admin (users/settings/billing/insights/golden/annotations/hit-test), SPA static hosting, `settings.rs` (hot-applied admin config), `matters.rs` , `annotations.rs`

Cross-cutting behaviour in the server:

- **Settings system** — admin-editable config persisted to `data/settings.json`: default query mode, per-surface system prompts (`ask`, `interact`, `draft`, `research_frame`, `research_adjudicate`, `agent`), LLM/embedding model names, brand, custom sidebar tabs. Prompt/mode/LLM changes apply **hot** (per request / `DynamicLlm` wrapper); the embedding model is boot-applied and requires restart + re-index (the UI warns).
- **Activity capture** — every user action recorded to `data/activity.json` (cap 5,000, FIFO) powering the Dashboard, history restore, golden-rule and annotation promotion. Private-vault content is redacted before logging.
- **Metering** — `usage.rs` records real provider token usage per call across query/stream/research/workflows/uploads; cache hits bill 0. Quota exhaustion returns HTTP 402.

5. Retrieval & generation pipeline

5.1 Ingest flow (LIVE-VERIFIED)

```
Upload (text / base64 file / S3 object)
  → parse: text layer (PDF) → typed content blocks (text/image/table/equation)
  → pages with insufficient text rendered (poppler) → Surya OCR (C5)
  → modal processors describe non-text blocks with extracted context
  → chunk: parent sections; each parent split into ~220-char children
     (sentence-aware incl. Devanagari danda) — children are the match units,
     parents are context units (LEGAL_RAG_PARENT_CHILD, LEGAL_RAG_CHILD_CHARS)
  → embed children (+ parents retained) with bge-m3, 1024-dim
  → LLM knowledge-graph extraction over parents (entities/relations)
  → index: vector store + BM25 corpus + document registry
  → DocStatus: pending → processing → ready / failed
```

5.2 Query flow (LIVE-VERIFIED)

```
Question
  1. Verified Answers check — curated reply at cosine ≥0.95, same scope +
     corpus fingerprint → served with zero LLM calls (never for web/private)
  2. Semantic answer cache — cosine ≥0.97, scope-keyed → replayed
     (stream hits replay Sources → Deltas → Done); never web/private
  3. Retrieval pool (three legs in parallel, tokio::join!):
       vector leg (wide: top_k×4 over children) + BM25 leg + graph/mode leg
       (LightRAG modes: naive/local/global/hybrid/mix/bypass; admin-set
       default = mix; brief scope hard-filters chunks + graph context).
       Legs merge by RANK INTERLEAVE — their score scales are
       incommensurable (cosines vs the graph legs' flat 0.85); a
       raw-score merge let graph chunks flood the pool and truncate the
       vector leg's best hits (measured root cause of several golden
       misses; fix verified 12/20 → 15/20, 23 Oct 2026). A retrieval-tier
       cache (fresh + stale-while-revalidate, corpus-fingerprint keyed)
       serves repeat retrievals.
  4. Fusion suite (vector↔BM25): RRF (k=60, default) | weighted RRF
     (per-leg weights, identifier boost) | DBSF — plus an in-fusion
     STATUTE TERM (a query naming an Act lifts that Act's chunks before
     the cut; production default ×1.35) and an optional recency-decay
     term → pool capped at 12 → optional Autocut score-gap cutoff
  5. Reranker (C4 cross-encoder, 500-char excerpts, 12s timeout,
     decisive-margin skip gates; vector order is the fallback)
  6. Parent expansion — matched children replaced by parent sections,
     siblings collapsed; citations point at parents
  7. Optional web leg (if ?web=true): primary-source probes + tiered search
     → two-wave fetch (top-3 protected wave) → best-passage extraction
  8. Private leg (if private=true and vault unlocked): owner's chunks only,
     top-3 decrypted in memory, [private:N] citations
  9. Generation (streaming SSE: heartbeat Status every 1.6s → Sources →
     Deltas → Done; verdict-first format addendum when web is blended)
 10. Citation-integrity audit — every [doc|web|private:N] marker checked
     against the exact excerpt shown to the model; <50% key-term support
     or unknown source → marker stripped; receipt returned to the client
 11. Metering + activity capture (+ answer-cache store when ≥1 citation)

Citation pinpoints (23 Oct 2026): each citation also carries the exact
sentence in the passage that supports the claim — selected by distinct
query-term coverage over flowing (de-wrapped) sentence segmentation
(English + Hindi danda), and only when the candidate is a complete
sentence (terminal punctuation, 40–700 chars). Cards display the
pinpoint; the source viewer highlights that sentence. Two retrieval
knobs are env-gated for corpus-scale tuning: `LEGAL_RAG_DEFAULT_TOP_K`
(request default, 5) and `LEGAL_RAG_PER_DOC_CAP` (per-document chunk cap
in the final context, off). Golden-set measurement on the 16-document
eval corpus found neither knob rescues ranking-level misses — the
rerank pool (step 4–5) is the binding constraint there.

### 5.3 Research & workflows

- **Research** (`research.rs`): frame hypotheses from the question → gather
  evidence per hypothesis (vault ± web) → adjudicate (IN FAVOUR / AGAINST /
  OPEN with rationale) → verdict-first report with citations. Prompts are
  admin-editable; overrides must preserve the JSON verdict contract.
- **Workflows** (`agent.rs`): fixed step pipelines — `contract_review` (5),
  `due_diligence` (4, incl. web), `memo_draft` (4, IRAC). Each step retrieves,
  generates, and cites; a final report synthesizes.
- **Autonomous agent** (`orchestrator.rs`, 23 Oct 2026): a planner–executor
  loop over the same tools — given a goal, the model plans one step at a
  time (search the vault, brief-scoped and space-checked, or search the
  web), up to a step budget (1–10, default 6), then synthesizes a
  verdict-first report from its findings only, with deduplicated
  citations. `POST /api/v1/knowledge/agents/autonomous`, billed like a
  workflow run. Honest limit: the planner is the main LLM — there are no
  separate sub-agent models.
```

6. Frontend architecture

Two independent Vite SPAs served as static files by C1 (no server restart
needed for frontend deploys — the server reads `dist` from disk per request
path config `LEGAL_RAG_FRONTEND_DIR` / `LEGAL_RAG_ADMIN_DIR`).

- **Design system** — REA-derived from Harvey's production UI: ivory `#fafaf9`,
  ink `#0f0e0d`, warm-grey scale, hairline borders, `rounded-lex` radii.
  Spec: `frontend/DESIGN.md`.
- **User app** — view-state routing in `AppShell` (Dashboard, Ask, Vault,
  Briefs, Interact, Draft, Research, Agents, Private Vault, Plans & Billing,
  Graph). Brief selection ("Working in") scopes Ask/Draft/Vault app-wide via
  a shared active-brief context. History restore rehydrates full results
  (answers + citations + sources sidebar) from the activity store.
- **Source viewer** — shared component (`components/SourceModal.tsx`,
  portal-rendered) used by Ask, Draft, Research, Agents: opens the full source
  document (via `POST /api/v1/knowledge/source-document`) with the cited
  passage highlighted (whitespace-normalized search, index-mapped back to
  original offsets, best-overlap fallback), provenance header (domain/date),
  and footer actions (Open original / Open in Interact / Private Vault).
- **Clause** — the assistant persona (avatar with live states) and the
  floating glass pet companion (draggable, expressions, page-aware tips,
  persisted position/size in `localStorage`). Pure frontend; no API impact.
- **Admin app** — separate bundle + route guard (admin role required);
  eight Command Center sections sharing one container.

7. Data architecture

7.1 Stores (current — interim, see ADR-0010)

All state lives under `LEGAL_RAG_DATA_DIR` as JSON, written atomically
(temp-file + rename). Snapshotter persists the RAG corpus every 5 minutes
and on SIGTERM/SIGINT; state restores on boot.

File	Contents	Sensitivity
`users.json`	Accounts, Argon2 hashes, roles (admin/user), plan	High
`documents.json` / `chunks.json` / `graph.json`	Corpus registry, chunk texts + vectors (in-memory mode), k
`matters.json`	Briefs: documents, timeline, memory notes, optional `space_id`, document legal-validity met
`spaces.json`	Multi-tenant Spaces: owner + members with roles (owner/editor/viewer)	Client data
`private_vault.json`	Private documents — **ciphertext only** + SHA-256 fingerprints	Highest (encrypted)
`billing.json`	Plans, subscriptions, payments (UTR refs), usage	Financial
`activity.json`	Action history (≤5,000 entries; private content redacted)	Medium
`annotations.json`	Verified Answers (curated Q/A + stored question vectors)	Medium
`golden.json`	Golden-ruleset evaluation cases	Low
`settings.json`	Admin settings + prompt overrides	Medium

7.2 Vector data

- Embedding space: **bge-m3, 1024 dimensions**, cosine similarity. Cross-
  lingual by design (Hindi query ↔ English document cosine ≈ 0.89–0.92).
- In-memory store: brute-force cosine over all records; corpus fingerprint
  (hash of doc set) versions every cache/annotation scope. Development
  fallback.
- SereneDB store (**production since 23 Oct 2026**, SereneDB 26.10.3):
  `FLOAT[1024]` column, IVF index
  (`USING inverted (embedding ivf (metric='cosine'))`), `<=>` operator
  (results cast to `float8` — the wire type is not directly decodable),
  Postgres wire protocol v3 on :7890, no auth by default (bind privately).
  The snapshotter restores into and saves from whichever store is active,
  so an externally-backed deployment boots with its corpus.
- Engine selection (`LEGAL_RAG_VECTOR_STORE`, ADR-0020 — Spice.ai's
  vector-engine lineup implemented natively behind the `VectorStore`
  trait): `hnsw` — pure-Rust in-process HNSW with the DuckDB-VSS
  engine's parameters (M=16, ef_construction=128, ef_search=64),
  JSON-file persistence, exact on small graphs; the embedded /
  Docker-tier engine (benched 20k×1024: 3.1 ms mean at recall@10
  0.899, 6.4 ms at 0.981, vs SereneDB IVF 67 ms at 1.000).

```
  `s3vectors` — AWS S3 Vectors (Spice's external engine) via a
  hand-rolled SigV4 client, `doc_id` as filterable metadata for
  predicate pushdown; the AWS-target engine (ap-south-1 available);
  live verification gated on AWS credentials.

## 8. Security architecture

| Layer | Mechanism (LIVE-VERIFIED unless noted) |
|---|---|
| Transport | HTTPS at the edge (relay today; ALB + ACM in target) |
| Authentication | Email/password (Argon2id hashes) → JWT access + refresh pair (`LEGAL_RAG_JWT_*_TTL`); optio
| Authorization | `User.role` (admin/user); `LEGAL_RAG_ADMIN_EMAILS` promotes at login/register; `AdminUser` e
| Brief confidentiality | Retrieval hard-filters to the brief's document set for chunks **and** graph context;
| Spaces (multi-tenancy) | Briefs may belong to a Space with owner/editor/viewer membership. Enforcement is se
| Legal-validity scoping | Documents carry jurisdiction / effective-from / effective-to / superseded-by metada
| Private Vault | Per-user password → Argon2id KDF → AES-256-GCM; key exists only in server memory while unloc
| Secrets | Provider keys + JWT secret in `.env` (gitignored) / Secrets Manager in target; never in code, logs
| Web fetching | Per-source failure isolation; fetch caps (10MB), timeouts; fetched content is data, never ins
| Payments | YouPI webhook verified with HMAC-SHA256; mock provider is test-only |

## 9. Billing architecture

- **Plans** (30-day periods, token-metered): Free 50,000 · Basic ₹2,999 /
  2,000,000 · Premium ₹9,999 / 10,000,000 tokens. No seat minimums.
- **Metering** counts real provider usage per operation; semantic-cache and
  verified-answer hits bill 0. Over-quota → HTTP 402 before generation.
- **Checkout**: `mock` (instant, test) or `youpi` — redirect to self-hosted
  YouPI-PG, poll for completion, HMAC-verified webhook upgrades the plan.
- **Admin**: MRR, collected totals, per-user spend, payment list with UTRs.
- **Risk note**: YouPI-PG is experimental software — see ADR-0013 before
  processing real payments on AWS.

## 10. Performance & measured capacity (21 Oct 2026)

| Metric | Value | Source |
|---|---|---|
| Healthy Ask latency (cached provider) | sources 2.6s · first word 3.6s · done 4.6s | live stream timing |
| Semantic-cache hit | 0.45s end-to-end | live |
| Web-blended Ask | ~36s (was 159s before concurrent fetch window) | live |
| Reranker quality | MRR 0.944 (vs 0.815 unranked) | offline bench, 12-q set |
| Hybrid retrieval | fused MRR 0.833, recall@3 1.0 | offline bench, core-9 |
| Parent-child (diluted corpus) | MRR 0.833 vs 0.639 parent-only | offline bench |
| OCR throughput | ~20 min/dense page on 2 vCPU — **GPU host required in production** | sandbox measurement |
| Rerank sidecar | ~9.5s per 10×500-char pool on 2 vCPU | sandbox measurement |
| Provider quota (free tier) | 10,000 neurons/day, resets 00:00 UTC (05:30 IST) | observed 429 / code 4006 |

Sandbox numbers are from a 2-vCPU dev box — treat as floor, not capacity
plan. AWS sizing guidance is in §12.4.

## 11. Reliability & failure modes

| Failure | Signature | System behaviour / recovery |
|---|---|---|
| LLM/embeddings quota exhausted | 429 (embeddings), code 4006 (LLM) after ~28s of retries | Friendly rate-lim
| SereneDB unreachable at boot | warning log | In-memory store; corpus rebuilt from snapshots |
| Reranker sidecar down | timeout (12s) | Vector order used; answers still grounded |
| Web source blocked/slow | per-source timeout (30s fetch, 45s wave) | Source skipped; DDG may anomaly-block d
| Process kill / host restart | — | Snapshots restore corpus + stores on boot; sidecars are stateless and rest
| Frontend deploy | — | Static files swapped on disk; no restart |

**Restore drill (dev sandbox)**: start C1 → relay agent → sidecars (rerank
~45s model load, embeddings ~85s) → test instance. In the AWS target this is
replaced by ECS service recovery + ALB health checks (ADR-0016).

## 12. Deployment architecture

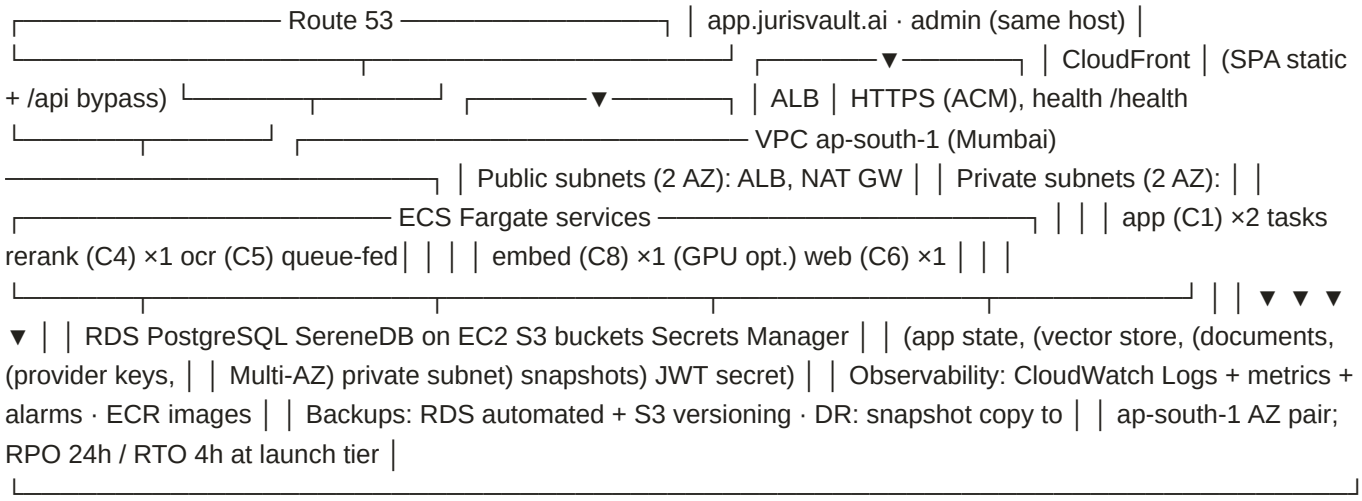
### 12.1 Today — development/demo (NOT production)
```

```
Single sandbox VM: C1 on :3000 (+ test instance on :3001 with an isolated
data copy and the Gemini bridge), sidecars on :8081/:8083, exposed publicly
through a Cloudflare Worker relay (C10). Purpose: demos and verification.
It has no SLA, restarts with the sandbox, and must not hold real client data.

### 12.2 Single-host Docker (small deployments)

`Dockerfile` (4-stage → `python:3.12-slim` runtime with poppler, non-root,
`/data` volume, healthcheck) + `docker-compose.yml` (app service; optional
`serenedb` profile). One command via `./deploy.sh`. **Status: authored and
validated, first Docker-host build pending** (ADR-0015).

### 12.3 AWS target topology (TARGET — for the AWS team)
```



12.4 AWS service mapping & sizing (launch tier)

Component	AWS service	Launch sizing	Notes
App server C1	ECS Fargate	2 tasks · 2 vCPU / 4 GB	Stateless once §12.5 lands; scale on CPU/requests
Reranker C4	ECS Fargate (CPU) or EC2 g5.xlarge (GPU)	1 task · 4 vCPU / 8 GB	GPU removes the ~9.5s/pool
OCR C5	ECS on EC2 GPU (g5) + SQS queue	1 node, autoscale 0–2	Surya is GPU-bound in production; queue d
Embeddings C8	ECS Fargate / GPU shared with C4	1 task	bge-m3 local = zero per-call provider cost + no
Web sidecar C6	ECS Fargate	1 task · 2 vCPU / 4 GB	Headless browser memory-bound
App state (target)	RDS PostgreSQL 16, Multi-AZ	db.t4g.medium	Replaces JSON stores (ADR-0010 migration)
Vectors	SereneDB on EC2 (private) **or** RDS pgvector	m6g.large	Decision in ADR-0006: SereneDB first (
Documents/snapshots	S3 (versioned, SSE-KMS)	—	Also keeps the S3 ingest source
Static SPAs	Served by C1 at launch; CloudFront+S3 later	—	Splitting is a config change, not a rewrite
Secrets	Secrets Manager	—	Injected as env vars (same names as Appendix A)
CI/CD	ECR + CodePipeline/GitHub Actions	—	`cargo` release build + two Vite builds → one image
Region	**ap-south-1 (Mumbai)**	—	Indian legal data residency; LLM provider calls leave region (disclos

12.5 Migration deltas for AWS (engineering checklist)

1. JSON stores → RDS PostgreSQL (schemas exist 1:1 from §7.1; keep the
   atomic-write repository layer as the seam). Vectors → SereneDB (§7.2).
2. Local `LEGAL_RAG_DATA_DIR` volume → EFS or S3-backed snapshot sync during
   transition; final state: no local state on tasks.
3. `.env` → Secrets Manager + ECS task env (same variable names).
4. Relay agent (C10) deleted; ALB health check on `GET /health`.
5. YouPI-PG: compliance review **or** replace with a mainstream PG (ADR-0013).
6. Cloudflare Paid **or** Bedrock/local models — provider decision (ADR-0008);
   the OpenAI-compatible client means this is config, not code.
7. First Docker build + load test on a Docker host (open since 20 Oct).

12.6 Alternative under verification — the shared AWS estimate (19 Oct 2026)

An AWS Pricing Calculator estimate was prepared and **already shared with the
AWS team on 19 Oct 2026**, before this document set existed. At the owner's
direction it is recorded here in full as a **deployment alternative** — the
AWS team is asked to verify it against §12.3/§12.4 and ADR-0016, **whether or
not it becomes the adopted deployment**. It is an EC2-centric design (single
API instance, Spot GPU ingest workers, dedicated SereneDB host), sized for a
16 TB corpus and a ~$2k/month budget, where §12.3 proposes ECS Fargate with
Multi-AZ state.

- **Estimate**: "My Estimate", AWS Pricing Calculator, dated 19 Oct 2026
  (pricing as of that date; read-only share)
- **Link**: https://calculator.aws/#/estimate?id=50a088f3e134c1dcb06525fc8e6b32c956e71512
- **Updated link (with WAF, 23 Oct 2026)**:
  https://calculator.aws/#/estimate?id=85ab95370800b927cdd6e0264e90946f86481e2d
  — in the calculator the WAF line shows $24.00/mo (total $2,039.78/mo),
  because its WAF form cannot model Bot Control's $10/mo subscription and
  per-request charges. This document keeps the published-pricing figure of
  $43.00/mo (total $2,058.78/mo) as canonical — owner decision, 23 Oct 2026.
- **Region**: Asia Pacific (Mumbai) for every line — consistent with ADR-0017
- **Totals**: upfront **$0.00** · monthly **$2,058.78** · 12 months
  $24,705.36 (USD, taxes excluded per AWS's own acknowledgement).
  The 13 line items as shared sum exactly to $2,015.78 (verified
  21 Oct 2026); line 14 (AWS WAF, $43.00) was added 23 Oct 2026 at the
  owner's direction — the shared estimate had priced no WAF — bringing
  the monthly total to $2,058.78.

#	Service	Configuration (as estimated)	$/mo
1	EC2	API server — m7i-flex.4xlarge, On-Demand, 730 h, ×1 ("scaled for $2k/mo budget")	588.38
2	EC2	GPU ingest worker — g4dn.xlarge, Spot (64% discount), ×1	152.16
3	EC2	Dedicated SereneDB — t3a.xlarge, On-Demand, 730 h, ×1	71.98
4	EBS	gp3, 2,218 GB total (API root 80 GB + SereneDB root 50 GB + SereneDB data 2,048 GB + GPU roots 2×2	
5	RDS for PostgreSQL	Metadata DB — db.t4g.small, **Single-AZ**, 40 GB gp3, On-Demand	35.90
6	S3	Source corpus — 16,384 GB Standard, 2M PUT/COPY/POST/LIST, 10M GET/other	423.60
7	Elastic Load Balancing	1 Application Load Balancer	20.37
8	S3	SereneDB nightly pg_dump backups — 1,024 GB, 7-day retention	25.61

```
| 9 | Secrets Manager | 1 secret, 10,000 API calls/mo | 0.45 |
| 10 | CloudWatch | 20 GB log ingestion, 5 standard alarms, 1 dashboard, OTEL events | 13.99 |
| 11 | Data Transfer | Internet egress 3,000 GB/mo | 327.90 |
| 12 | ECR | Container images, 10 GB stored | 1.00 |
| 13 | EC2 | GPU ingest worker #2 — g4dn.xlarge, Spot, ×1 (parallel ingest for 16 TB) | 152.16 |
| 14 | AWS WAF | Web ACL on the ALB (added 23 Oct 2026) — 1 Web ACL ($5.00); 6 charged rules ($6.00): AWS Mana
| | **Total** | | **2,058.78** |

Design notes and the verification questions this raises for the AWS team:

- **Topology contrast**: EC2 hosts with named roles vs ADR-0016's Fargate
  services. This estimate has **no HA** for the API (one instance) and a
  **Single-AZ** RDS; ADR-0016 proposes 2 app tasks and Multi-AZ. Verify
  which availability posture production actually requires.
- **The 16 TB corpus assumption drives $649.88/mo** (S3 corpus $423.60 +
  SereneDB data volume inside EBS) — confirm the real corpus size at launch;
  the current production corpus is three documents. Right-sizing this line
  is the single largest saving available.
- **Egress is the second-largest cost** ($327.90 for 3 TB/mo at "thousands
  of users/day"). Verify the traffic assumption; also check for double
  counting — the EC2 rows select 0 TB data transfer while this separate
  Data Transfer line carries the full 3 TB.
- **Spot GPU workers** (2× g4dn.xlarge at a 64% Spot discount) fit the
  queue-fed OCR/ingest design (ADR-0007) well, but Spot interruption
  handling must be designed (checkpointed ingest jobs); the estimate prices
  them as "consistent" workloads.
- **Gaps to price or rule out**: no CloudFront, no NAT Gateway (WAF was
  priced as line 14 on 23 Oct 2026 — its 20M requests/mo planning figure
  should be reconciled with the 3 TB egress assumption), EC2
  detailed monitoring disabled (CloudWatch is estimated separately), no
  backup cost for the API/SereneDB EBS volumes beyond the S3 pg_dump line,
  and no LLM/provider spend (ADR-0008 — correctly outside AWS, but it
  belongs in the same budget conversation).
- Line descriptions still carry the legacy pre-rename product name (the
  estimate predates the Juris Vault AI rename) — cosmetic only.

**Status**: recorded as an alternative under verification (owner direction,
21 Oct 2026). Adoption is a separate decision, taken with ADR-0016.

## 13. Observability

- **Today**: structured stdout logs per service; `/health` on C1 and each
  sidecar; admin Insights (usage, top questions, 14-day activity);
  `/api/v1/knowledge/cache-stats` (cache hit economics); activity store as an
  audit trail.
- **Target**: CloudWatch Logs (one log group per ECS service), metrics on
  query latency/audit strip-rate/quota headroom, alarms on 5xx + provider
  429s, X-Ray or OTEL traces across the query pipeline stages (§5.2 steps are
  natural span boundaries).

## 14. Testing & benchmarks

- **Unit/integration**: 70 Rust workspace tests (retrieval, audit incl. the
  polluted-citation case, cache scoping, vault crypto, workflows) — green.
- **Offline benches** (reproducible scripts + reports retained): reranker
  head-to-head (bge vs LLM judge vs mxbai vs gte), OCR head-to-head (Surya vs
  Tesseract vs Paddle — Paddle unbenchable, vendor crash, recorded), BM25
  fusion, parent-child dilution, cross-lingual cosine checks.
- **Golden ruleset**: admin-curated Q&A with expected terms + sources, scored
  against live answers (`POST /admin/golden/run`).
- **Hit-testing**: admin debugger shows vector/BM25/fused/reranked stages per
  query — the first tool when a user reports a bad answer.

## 15. Limitations & roadmap (summary)

**Limitation — provider quota only:** the production provider is
Cloudflare Workers AI's free tier, capped at 10,000 neurons/day (reset
00:00 UTC); the Paid plan (~$5/mo) or a provider move removes it
```

(ADR-0008, decision open). **Roadmap:** JSON persistence → managed
PostgreSQL on the AWS target (§12.5, ADR-0010/0016); first Docker image
build. Everything else formerly listed here — sentence-level citations,
jurisdiction/temporal awareness, multi-tenant Spaces, autonomous agents,
and the fusion suite / leg-merge fix (golden 12/20 → 15/20, measured
23 Oct 2026) — has shipped and is described in §5/§8.

Appendix A — Configuration reference

Server (C1) environment. Defaults in `crates/common` / `.env.example`.

Core: `LEGAL_RAG_ADDR` (bind), `LEGAL_RAG_PORT` (compose), `LEGAL_RAG_DATA_DIR`,
`LEGAL_RAG_FRONTEND_DIR`, `LEGAL_RAG_ADMIN_DIR`, `LEGAL_RAG_USE_MOCK_PROVIDERS`.
Auth: `LEGAL_RAG_JWT_SECRET`, `LEGAL_RAG_JWT_ACCESS_TTL`, `LEGAL_RAG_JWT_REFRESH_TTL`,
`LEGAL_RAG_ADMIN_EMAILS`, `LEGAL_RAG_OIDC_ISSUER`, `LEGAL_RAG_OIDC_CLIENT_ID`,
`LEGAL_RAG_OIDC_CLIENT_SECRET`, `LEGAL_RAG_OIDC_REDIRECT_URI`.
Providers: `LEGAL_RAG_LLM_URL`, `LEGAL_RAG_LLM_API_KEY`, `LEGAL_RAG_LLM_MODEL`,
`LEGAL_RAG_EMBEDDING_URL`, `LEGAL_RAG_EMBEDDING_API_KEY`, `LEGAL_RAG_EMBEDDING_MODEL`,
`LEGAL_RAG_EMBEDDING_DIMS`.
Retrieval: `LEGAL_RAG_RERANKER` (`local`|`llm`), `LEGAL_RAG_RERANK_URL`,
`LEGAL_RAG_RERANK` (on/off), `LEGAL_RAG_RERANK_CANDIDATES`, `LEGAL_RAG_RERANK_POOL`,
`LEGAL_RAG_RERANK_TIMEOUT_MS`, `LEGAL_RAG_CACHE_ENABLED`, `LEGAL_RAG_SEMANTIC_CACHE`,
`LEGAL_RAG_SEMANTIC_THRESHOLD`, `LEGAL_RAG_ANNOTATION_THRESHOLD`,
`LEGAL_RAG_PARENT_CHILD`, `LEGAL_RAG_CHILD_CHARS`, `LEGAL_RAG_CHUNK_SIZE`,
`LEGAL_RAG_CHUNK_OVERLAP`.
Vector store: `SERENEDB_URL`, `SERENEDB_TABLE`.
Ingest/OCR: `LEGAL_RAG_S3_BUCKET`, `LEGAL_RAG_S3_REGION`, `LEGAL_RAG_S3_PREFIX`,
`RAG_ANYTHING_MAX_CONCURRENT_FILES`, `RAG_ANYTHING_ENABLE_IMAGE_PROCESSING`,
`RAG_ANYTHING_ENABLE_TABLE_PROCESSING`, `RAG_ANYTHING_ENABLE_EQUATION_PROCESSING`,
`RAG_ANYTHING_OCR_ENABLED`, `RAG_ANYTHING_OCR_MIN_TEXT_CHARS`,
`RAG_ANYTHING_OCR_DPI`, `RAG_ANYTHING_OCR_PRIMARY_CMD`.
Web: `LEGAL_RAG_WEB_SEARCH`, `LEGAL_RAG_WEB_SIDECAR`.
Billing: `LEGAL_RAG_PAYMENT_PROVIDER` (`mock`|`youpi`), `LEGAL_RAG_YOUPI_URL`,
`LEGAL_RAG_YOUPI_MERCHANT_ID`, `LEGAL_RAG_YOUPI_WEBHOOK_SECRET`.

Appendix B — API inventory (65 routes, `/api/v1`)

- **auth**: register · login · refresh · me · oidc/login · oidc/callback
- **vault**: documents (create/list/delete) · search · ingest (S3)
- **knowledge**: query · query/stream (SSE) · research · workflows ·
  workflows/run · interact · draft · source-document · graph · stats ·
  cache-stats · documents (interact corpus)
- **matters**: briefs CRUD · documents attach/detach · timeline · memory
- **billing**: plans · me · checkout · payments · payments/:id · webhook/youpi
- **private-vault**: status · password (set) · unlock · lock · documents
- **settings**: public · **dashboard**: me
- **admin**: users (list/get/patch) · settings (get/put) · billing/overview ·
  billing/settings · insights · insights/users/:id · golden/rules (+from-activity) ·
  golden/run · annotations (+from-activity, toggle) · hit-test

Appendix C — Port & process inventory

Port	Process	Env
3000	legal-rag-server (prod instance)	production
3001	legal-rag-server (test instance, `data-test`, Gemini bridge LLM)	test only
7890	SereneDB	optional
8081	reranker sidecar	all
8082	Gemini bridge	test only
8083	local bge-m3 embeddings sidecar	test/local

---

*Document control: v1.2 — authored 21 Oct 2026 by the Juris Vault AI Team (v1.1 adds §12.6, the shared-estimat
(owner: Jishan). Update this document in the same change as any architecture

```
change, and add the corresponding ADR (or amend the existing one) — see the
changelog discipline in [CHANGELOG.md](../CHANGELOG.md).*
```

PART II

Architecture Decision Records

Document	Full ADR set for the infrastructure and AWS platform teams
Product	Juris Vault AI — legal document intelligence for Indian law practice
Owner	**Jishan** — Juris Vault AI Team
Version	1.6 — 23 October 2026
Companion	ARCHITECTURE.md · README.md · CHANGELOG.md

How to read this document

Each record follows the same shape: **Status · Context · Decision · Consequences · Alternatives considered**. Statuses:

- **Accepted** — decided and implemented (marked LIVE-VERIFIED where proven).
- **Accepted with open action** — decided; a named follow-up remains.
- **Proposed** — designed in full, awaiting the owner's (Jishan's) decision. Infra/AWS teams should build foundations that keep Proposed records cheap to adopt, but must not treat them as approved spend.

Records are immutable once accepted: a changed decision gets a **new** ADR that supersedes the old one (the old record stays, marked Superseded).

Index

ADR	Title	Status
0001	Rust monorepo: five-crate workspace, single-binary server	Accepted
0002	RAG architecture: faithful RAG-Anything port + LightRAG query modes	Accepted
0003	Embeddings: bge-m3 fixed; improve via a decision layer, not a swap	Accepted
0004	Reranker: bge-reranker-v2-m3 local sidecar as the decision layer	Accepted
0005	Hybrid retrieval: vector + BM25 with RRF fusion	Accepted
0006	Vector store: in-memory default, SereneDB persistent backend	Accepted
0007	OCR: Surya-only pipeline	Accepted
0008	LLM provider: Cloudflare Workers AI today; provider abstracted	Accepted with open action
0009	Parent-child chunking for match/context separation	Accepted
0010	Persistence: atomic JSON stores now; PostgreSQL migration path	Accepted
0011	Auth: Argon2 + JWT, role-based admin, optional OIDC	Accepted
0012	Confidential Vault: Argon2id → AES-256-GCM, memory-only key	Accepted
0013	Billing: token metering; mock + YouPI-PG payments	Accepted with open action
0014	Web research: self-hosted sidecars, primary-source-first, citation audit	Accepted
0015	Deployment today: single host / Docker; sandbox relay is dev-only	Accepted with open action

ADR	Title	Status
0016	AWS target architecture: ECS Fargate + RDS + ap-south-1	**Proposed**
0017	Data residency & compliance posture: India-first	Accepted
0018	Adopt functionality, never identity, from reference products	Accepted

ADR-0001 — Rust monorepo: five-crate workspace, single-binary server

Status: Accepted (LIVE-VERIFIED)

Context. The system needs a low-latency streaming API, heavy in-process retrieval state, and simple deployment. The founding brief specified a Rust implementation end to end (auth through serving).

Decision. One Cargo workspace (resolver 2, edition 2021) with five crates — `common` (config/errors), `auth`, `ingest` (S3), `rag` (intelligence core), `server` (Axum API + static SPA hosting) — compiled to one release binary, `legal-rag-server`. The server serves both frontends from disk, so frontend deploys never restart the API. Sidecars (rerank/OCR/web) are separate processes by design (ADR-0004, 0007, 0014), not crates.

Consequences. (+) One artifact to ship, version, and roll back; shared types across API and retrieval; predictable memory profile. (−) Rust build times; a bug in any route module ships in the same binary (mitigated by the 70-test workspace suite and crate boundaries).

Alternatives considered. Node/TypeScript service (rejected: the brief mandated Rust, and the retrieval core benefits from Rust's concurrency and memory control); microservices per surface (rejected: operational weight far beyond the team's current scale).

ADR-0002 — RAG architecture: faithful RAG-Anything port + LightRAG modes

Status: Accepted (LIVE-VERIFIED)

Context. Rather than inventing a retrieval architecture, the build ports two proven open-source designs: **RAG-Anything** (document pipeline: parse → typed content blocks → modal processors → index) and **LightRAG** (six query modes — `naive`, `local`, `global`, `hybrid`, `mix`, `bypass` — over a knowledge graph + vector hybrid).

Decision. Port both faithfully in the `rag` crate: same config shape (`RagAnythingConfig`), same pipeline stages, same mode semantics. Product-, India-, and provenance-specific layers (parent-child chunking, ADR-0009; citation audit, ADR-0014; Brief scoping) are added **on top of** the port, never by distorting it.

Consequences. (+) Behaviour is comparable against upstream references; upstream improvements remain portable. (+) Modes give the admin a real retrieval dial (default exposed to users is `mix`; the picker itself is hidden from end users — a product decision recorded in the changelog). (−) Port fidelity constrains some local optimizations.

Alternatives considered. LangChain/LlamaIndex in Python (rejected with ADR-0001); building only on a hosted RAG API (rejected: no control over chunking, provenance, or Indic language behaviour).

ADR-0003 — Embeddings: bge-m3 fixed; improve via a decision layer

Status: Accepted (LIVE-VERIFIED)

Context. Retrieval quality for Indian legal text is dominated by two needs: Hindi↔English cross-lingual matching and precise ranking among near-duplicate statutory chunks. The owner directed (20 Oct 2026): keep **bge-m3** as the

embedding model; add a "decision model" layer instead of swapping embeddings.

Decision. Embeddings are `BAAI/bge-m3`, 1024-dim, cosine space, for all ingest and query paths (provider-hosted today; identical model in the local sidecar, ADR-0008/0016). Quality investment goes into the reranking decision layer (ADR-0004). Measured cross-lingual quality: Hindi query ↔ English document cosine 0.89 (hosted) / 0.919 (local sidecar).

Consequences. (+) Stable vector space — no corpus re-embedding on model fashion changes; the local sidecar can replace hosted embeddings with zero drift (verified: local vectors retrieve correctly against the hosted-embedded corpus). (−) We forgo potential gains from newer embedding models until a head-to-head justifies a full re-index.

Alternatives considered. Swapping to a newer multilingual embedding model (deferred by owner direction; requires full re-embed + re-bench); OpenAI embeddings (rejected: weaker Indic cross-lingual evidence, per-call cost, data leaves India).

ADR-0004 — Reranker: bge-reranker-v2-m3 local sidecar as decision layer

Status: Accepted (LIVE-VERIFIED)

Context. ADR-0003's decision layer needed a decider. Candidates were benchmarked offline on a fixed legal query set (report retained).

Decision. `BAAI/bge-reranker-v2-m3` runs as a local HTTP sidecar (:8081), scoring the fused pool (cap 12 candidates, 500-char excerpts, 12s timeout, decisive-margin skip gates; vector order is the automatic fallback). Benchmarks: **MRR 0.944** vs 0.815 unranked baseline; the LLM listwise judge tied on quality at per-call cost, so it remains an option (`LEGAL_RAG_RERANKER=llm`), not the default. Round-2 challengers (closed 21 Oct 2026): mxbai-rerank-base-v2 MRR 0.889 — does not dethrone; gte-multilingual-reranker-base unbenchable (crashes in vendor modeling code).

Consequences. (+) Best measured quality at zero marginal cost; no quota exposure. (−) A GPU-less host pays ~9.5s per 10-candidate pool (2-vCPU sandbox) — the AWS sizing for this component is an open benchmark item in ADR-0016. **License note:** Jina reranker weights were rejected (CC BY-NC — commercial trap); bge-reranker-v2-m3 is MIT.

Alternatives considered. LLM-as-judge default (kept as option); Cohere Rerank API (hosted cost + data egress; not benchmarked better).

ADR-0005 — Hybrid retrieval: vector + BM25 with RRF fusion

Status: Accepted (LIVE-VERIFIED)

Context. Pure vector retrieval misses exact statutory terms (section numbers, defined terms). A BM25 leg was added and fused with Reciprocal Rank Fusion (k=60).

Decision. Retrieval pool = RRF fusion of the vector leg (wide, top_k×4) and a BM25 leg (k1=1.5, b=0.75, non-negative IDF) over chunk texts, with the graph/mode leg per ADR-0002; the three legs run concurrently (`tokio::join!`). Keyword-only hits receive a rank-imputed cosine so all downstream margin gates stay on one scale. Offline bench: fused **MRR 0.833** vs 0.815 vector-only, recall@3 1.0, no query regressed. The BM25 index rebuilds lazily on corpus-fingerprint change; `LEGAL_RAG_BM25=0` disables.

Consequences. (+) Exact-term recall (section numbers!) without losing semantic recall. (−) One more in-memory index to keep fingerprint-fresh.

Alternatives considered. Elasticsearch/OpenSearch sidecar (rejected at current scale: operational weight; revisit if corpus passes ~1M chunks).

ADR-0006 — Vector store: in-memory default, SereneDB persistent backend

Status: Accepted (**LIVE-VERIFIED in production since 23 Oct 2026** — SereneDB 26.10.3 runs the production corpus; the gated live test initially failed on the real server's distance-column type and passes with a `float8` cast on `<=>` results, and the snapshot restore/save path was fixed to honor whichever store is active. SereneDB ships Docker-only, so the sandbox runs the official image's statically-linked `serened` natively. The remaining open question is narrowed to the AWS target: SereneDB (self-hosted) vs RDS pgvector — see the decisions register.)

Context. Development needs zero-dependency startup; production needs durable, indexed vectors. **SereneDB** (serenedb.com, Apache 2.0) speaks the Postgres wire protocol and supports native vector columns, and was the owner's pick over the in-memory default.

Decision. A `VectorStore` trait with two backends. Default: in-memory brute-force cosine (corpus restored from JSON snapshots). With `SERENEDB_URL` set: SereneDB — `FLOAT[N]` columns, `<=>` cosine operator, IVF index (`USING inverted (embedding ivf (metric='cosine'))`), Postgres wire v3 on :7890; unreachable at boot → automatic fallback to in-memory with a warning.

Consequences. (+) Same code path in dev and prod; swap is one env var. (−) SereneDB is a young project (Berlin, 2025) — the AWS record (ADR-0016) keeps RDS **pgvector** as the documented fallback if SereneDB fails its production evaluation. (−) SereneDB ships Docker-only; no standalone binary.

Alternatives considered. pgvector on RDS (fallback, above); Pinecone/ Qdrant hosted (rejected for residency and cost at this stage); keeping vectors in Postgres JSON (rejected: no index, no scale path).

ADR-0007 — OCR: Surya-only pipeline

Status: Accepted (LIVE-VERIFIED)

Context. Indian legal corpora are full of scanned paper — court notices, gazette prints, Hindi documents. OCR quality decides whether those documents exist for retrieval at all.

Decision. The PDF pipeline is: text layer first (fast path) → rendered pages (`pdftoppm`) → **Surya 0.22 as the sole OCR engine**, via a local sidecar (llama.cpp inference backend). A head-to-head benchmark (Surya vs Tesseract 5 on English + Hindi + ruled tables) showed Surya tying or winning everywhere and winning decisively on Hindi and tables; per the owner's direction Tesseract was removed entirely (20 Oct 2026), and the Docling fallback was removed after it. Operational gotchas (silent empty output when the llama.cpp backend fails to spawn; results-path nesting) are documented in the team's runbook notes.

Consequences. (+) One OCR behaviour to tune, verify, and support; best measured Indic quality. (−) Surya is slow on CPU (~20 min for a dense page on 2 vCPU) — **production OCR requires a GPU host**; this is a hard input to ADR-0016 sizing. (−) PaddleOCR was evaluated as a challenger and is unbenchable in our environment (crashes inside Paddle's own executor) — recorded as an infrastructure incompatibility, not a quality verdict.

Alternatives considered. Tesseract (removed, above); cloud OCR APIs (Textract/Google Vision — per-page cost, data egress, weaker Devanagari evidence in our tests).

ADR-0008 — LLM provider: Cloudflare Workers AI today; provider abstracted

Status: Accepted with open action (LIVE-VERIFIED; **owner decision pending** on the production provider)

Context. Generation, graph extraction, research adjudication, drafting, and workflows all need an LLM. All LLM/embedding calls go through OpenAI-compatible HTTP clients (`LEGAL_RAG_LLM_*`, `LEGAL_RAG_EMBEDDING_*`), so the provider is configuration, not code.

Decision. Today: **Cloudflare Workers AI** — LLM `@cf/meta/llama-3.3-70b-instruct-fp8-fast`, embeddings `@cf/baai/bge-m3`. Model selection is admin-changeable and hot-applied (`DynamicLlm`); a Gemini bridge (anonymous gemini-web2api → Gemini Flash) powers only the isolated test instance. A local bge-m3 embeddings sidecar already removes embeddings from the provider entirely where deployed.

The open action (owner: Jishan). The Cloudflare **free tier allows 10,000 neurons/day** (reset 00:00 UTC); exhaustion presents as embedding 429s and LLM error 4006. Production must pick one: (a) Cloudflare Workers **Paid** (~$5/mo + usage), (b) **AWS Bedrock** models in-region where available, or (c) self-hosted models on the GPU node from ADR-0016. Heavy verification testing burned a full day's free quota on 21 Oct 2026 — treat free-tier quota as a demo resource only.

Consequences. (+) Provider portability is real (three providers already exercised: Cloudflare, Gemini bridge, mock). (−) Until the decision lands, production capacity planning has an unknown.

Alternatives considered. OpenAI direct (works via the same client; weaker India-residency story); locking to any single provider in code (rejected outright).

ADR-0009 — Parent-child chunking for match/context separation

Status: Accepted (LIVE-VERIFIED)

Context. Long statutory sections chunk poorly: match on a whole section and precision drowns; match on fragments and the model lacks context — the classic failure that Dify's parent-child design addresses (adopted as an idea, ADR-0018).

Decision. Parents = document sections; each parent is split into ~220-char sentence-aware children (Devanagari danda respected; `LEGAL_RAG_CHILD_CHARS`, disable with `LEGAL_RAG_PARENT_CHILD=0`). Children are the match units (embeddings, BM25); after reranking, matches expand to their parents for generation; citations point at parents; sibling matches collapse. Bench: diluted corpus MRR **0.833 (child) vs 0.639 (parent-only)**. Live-verified on the test instance (buried clause retrieved via child, answered from parent).

Consequences. (+) Precision and context at once. (−) ~4–5× record count vs parent-only indexing; legacy documents gain children only on re-ingest.

Alternatives considered. Fixed-size single-level chunking (the bench above is the rejection evidence); semantic chunking via LLM (cost/latency, deferred).

ADR-0010 — Persistence: atomic JSON stores now; PostgreSQL migration path

Status: Accepted (LIVE-VERIFIED for JSON; migration is TARGET)

Context. The system launched in a sandbox with no managed database. State had to be durable, inspectable, and restorable across restarts.

Decision. All state is JSON files under `LEGAL_RAG_DATA_DIR`, written atomically (temp + rename): users, documents/chunks/graph, matters, billing, activity (≤5,000 entries), annotations, golden rules, settings, snapshot corpus (every 5 min + on SIGTERM/SIGINT, restored on boot). The data layer is kept behind repository-style modules so the AWS migration (ADR-0016) swaps storage, not callers: **target = RDS PostgreSQL** (schemas map 1:1 from the §7.1 inventory in ARCHITECTURE.md), vectors per ADR-0006.

Consequences. (+) Zero-dependency durability today; every store is human-inspectable during incidents. (−) Single-writer assumption — horizontal app scaling is blocked until the RDS migration lands; file stores cap practical corpus/tenant growth. Both are accepted for launch scope and are the first migration items in ADR-0016 §12.5.

Alternatives considered. SQLite (rejected: no better scaling story, worse inspectability for JSON-shaped state); going straight to Postgres in the sandbox (no managed instance available; would have slowed the build phase).

ADR-0011 — Auth: Argon2 + JWT, role-based admin, optional OIDC

Status: Accepted (LIVE-VERIFIED)

Context. The reference product (Harvey) gates everything behind an Auth0 login wall. The product needs the same posture with a self-hosted default and an enterprise SSO path.

Decision. Email/password with **Argon2id** hashes; **JWT access + refresh** pairs (TTLs via `LEGAL_RAG_JWT_*_TTL`, secret via `LEGAL_RAG_JWT_SECRET`); authorization by `User.role` with an `AdminUser` extractor (403 for non-admins) and `LEGAL_RAG_ADMIN_EMAILS` promotion at login/register; an **Auth0-compatible OIDC** client (login/callback) ships in the `auth` crate for tenants who bring their own IdP — enabling a real Auth0 tenant is a go-live configuration task, not code.

Consequences. (+) No external dependency for the default path; SSO is a config flip. (−) Password auth means we own credential hygiene (Argon2id, rate-limiting at the edge in target topology).

Alternatives considered. OIDC-only (rejected: blocks small-firm onboarding); session cookies (rejected: the API is consumed by two SPAs and future integrations; JWT fits).

ADR-0012 — Confidential Vault: Argon2id → AES-256-GCM, memory-only key

Status: Accepted (LIVE-VERIFIED)

Context. Firms will store genuinely confidential client documents. A "private" flag in a shared corpus is not confidentiality.

Decision. Per-user vault password → **Argon2id** KDF → **AES-256-GCM**. Document titles and chunk texts are stored **only as ciphertext** (`private_vault.json`) with SHA-256 fingerprints; the key lives only in server memory while unlocked; **there is no recovery by design**. Private ingest never enters the shared corpus, snapshots, or knowledge graph; query with `private: true` embeds the question, searches only the owner's chunks, decrypts the top 3 in memory, and cites them as `[private:N]`; a locked vault returns an explicit 400. Activity logs redact private content. Verified: the owner retrieves the fact with citation; another user retrieves nothing; on-disk grep finds zero plaintext.

Honest limits (accepted, disclosed in-product). While unlocked, decrypted context is sent to the LLM to answer; embeddings are computed by the embedding provider at upload; embedding vectors are stored beside the ciphertext. Client-side encryption (zero-knowledge) is roadmap, not claimed.

Consequences. (+) Real at-rest confidentiality with a small, auditable crypto surface. (−) Forgotten vault password = unrecoverable data (by design; the UI must keep saying so).

Alternatives considered. Corpus-level row security (rejected: plaintext at rest); per-document passwords (rejected: unusable UX, same crypto).

ADR-0013 — Billing: token metering; mock + YouPI-PG payments

Status: Accepted with open action (LIVE-VERIFIED with mock + webhook flow; **compliance decision pending before real money**)

Context. Pricing research placed the product in the ₹1k–9k/month research band that incumbents leave open. Billing must meter what actually costs money — provider tokens.

Decision. Token-metered plans on 30-day periods: **Free 50k tokens · Basic ₹2,999 / 2M · Premium ₹9,999 / 10M**, no seat minimums. Metering uses real provider usage across query/stream/research/workflows/uploads; semantic-cache and verified-answer hits bill 0; over-quota requests fail with HTTP 402 before generation. Checkout providers: `mock` (testing) and **YouPI-PG** (self-hosted UPI gateway, 0% fees, HMAC-SHA256-verified webhook).

Open action. YouPI-PG is self-described experimental/research software. Before real payments on AWS: compliance review + bank reconciliation process, **or** swap to a mainstream gateway (Razorpay/Stripe) behind the same checkout/webhook seam. Owner decision (Jishan), infra to implement.

Consequences. (+) Pricing maps to cost; the meter is provider-agnostic. (−) UPI-only today (cards/netbanking arrive with the mainstream gateway).

Alternatives considered. Seat-based pricing (rejected: misaligned with usage cost and the target band); Stripe-only (international-first; UPI is the Indian default rail).

ADR-0014 — Web research: self-hosted sidecars, primary-source-first, citation audit

Status: Accepted (LIVE-VERIFIED)

Context. Legal answers that cite news write-ups of a judgment instead of the judgment are a credibility failure — observed directly in a competitor's output (a captured answer cited an unrelated Karnataka case for the central holding of the electoral-bonds judgment). Web blending therefore ships with three guardrails or not at all.

Decision.

1. **Self-hosted search/fetch sidecars** (Python, `tools/web`): DDG-HTML + Bing-HTML over plain HTTP (~3s), merged, canonical-URL deduped; stealth Firefox fallback when result-starved (DDG anomaly-blocks datacenter IPs after bursts — observed HTTP 202s); fetch is HTTP-first with **pypdf extraction for judgment PDFs**, meta-tag dates, browser fallback.
2. **Primary-source-first**: legal-authority tiers (T1: sci.gov.in, indiankanoon.org, ecourts, gazettes, indiacode; T2: LiveLaw, Bar & Bench, quality press) + explicit `site:` probes for judgment/statute queries; tier order floats primary authorities to the top of the model's context. Two-wave fetching protects the top 3 with a dedicated 45s window.
3. **Citation-integrity audit** (`rag/src/audit.rs`): every `[doc|web|private:N]` marker in a generated answer is checked against the exact excerpt the model was shown; <50% key-term support or an unknown source → the marker is stripped; the client receives the receipt (production observation: 12 checked / 9 verified / 3 stripped). Verdict- first answer format is enforced when web sources are blended. Supporting UX: the **source viewer modal** opens the full source document with the cited passage highlighted (Ask, Draft, Research, Agents).

Consequences. (+) Web answers cite the actual judgment (verified: the electoral-bonds question returns the SC judgment PDF from api.sci.gov.in as source #1). (+) Unsupported citations never reach the user. (−) Web Ask is slower (~36s) than vault Ask (~5s); some publishers (LiveLaw) defeat both fetch paths — tolerated per-source.

Alternatives considered. Tavily/Serper APIs (rejected: cost, egress, and no tier control); LLM-parametric "web knowledge" without fetching (rejected outright — unverifiable).

ADR-0015 — Deployment today: single host / Docker; sandbox relay is dev-only

Status: Accepted with open action (LIVE-VERIFIED for the sandbox runtime; Docker path BUILT, first build pending)

Context. Development and demos run in a sandbox VM with no Docker and no public ingress. A portable deployment artifact was still required for handover.

Decision. Canonical runtime = the release binary + sidecars on one host (or the Docker image). The sandbox demo is exposed via a Cloudflare Worker relay + local agent (`tools/tunnel`) — **explicitly a development convenience, never a production component** (it disappears in ADR-0016). The Docker path: 4-stage `Dockerfile` (→ `python:3.12-slim`, poppler, non-root, `/data` volume, healthcheck), `docker-compose.yml` (app + optional `serenedb` profile), `deploy.sh`, `DEPLOY.md`.

Open action. First `docker build` on a real Docker host + smoke test (pending since 20 Oct 2026 — the sandbox has no Docker). Infra team: treat this as go-live task #1.

Consequences. (+) Handover is one command once Docker exists. (−) Until the first build, the image is validated (YAML/bash) but unproven.

Alternatives considered. Kubernetes for the dev phase (rejected: disproportionate); bare-metal-only docs (rejected: compose is the portable unit the AWS migration also starts from).

ADR-0016 — AWS target architecture (PROPOSED)

Status: Proposed — awaiting owner (Jishan) approval. Addressed to the AWS team: this is the build spec to estimate and stage, not yet approved spend.

Context. Production needs: an SLA, GPU capacity for OCR (ADR-0007), managed state (ADR-0010), secrets management, India data residency (ADR-0017), and an end to sandbox restarts and provider free-tier walls (ADR-0008).

Decision (proposed). Region **ap-south-1 (Mumbai)**, two AZs.

- **Ingress**: Route 53 → CloudFront (SPA static; `/api` bypass) → ALB (HTTPS via ACM, health check `GET /health`).
- **Compute**: ECS Fargate services — `app` ×2 tasks (2 vCPU/4 GB), `rerank` ×1, `web` ×1, `embed` ×1 (local bge-m3, ADR-0003); **OCR on ECS/EC2 GPU (g5) behind an SQS queue**, autoscale 0–2. GPU for rerank/embed is a benchmark decision at staging (CPU numbers in ARCHITECTURE §10).
- **Data**: RDS PostgreSQL 16 Multi-AZ (db.t4g.medium at launch) for app state per ADR-0010; **SereneDB on EC2 (private subnet)** for vectors per ADR-0006, with pgvector as the documented fallback; S3 (versioned, SSE-KMS) for documents and snapshots; the existing S3 ingest source is unchanged.
- **Config/secrets**: Secrets Manager → ECS task environment, **same variable names** as ARCHITECTURE Appendix A. No `.env` files in production.
- **Images/CI**: ECR + pipeline (cargo release build + both Vite builds → one image per ADR-0015); blue/green via ECS deployment circuit breaker.
- **Observability**: CloudWatch Logs per service, alarms on 5xx and provider 429 rates, dashboards for query latency + audit strip-rate + quota headroom (or provider spend under the ADR-0008 decision).

- **Backup/DR (launch tier)**: RDS automated backups + S3 versioning; RPO 24h / RTO 4h; cross-region copy deferred until the first enterprise contract requires it.
- **Relay agent (C10) is deleted**, not migrated.

Migration order (from ARCHITECTURE §12.5): Docker first build → RDS schemas from the JSON stores → staging in ap-south-1 with the local embeddings sidecar (kills the free-tier wall) → GPU OCR benchmark → provider decision (ADR-0008) → production cutover → YouPI decision (ADR-0013) before real payments.

Alternative under verification (added 21 Oct 2026, owner direction). An AWS Pricing Calculator estimate prepared and shared with the AWS team on **19 Oct 2026** — EC2-centric (m7i-flex.4xlarge API host, 2× Spot g4dn.xlarge GPU ingest workers, dedicated t3a.xlarge SereneDB host, Single-AZ db.t4g.small RDS, 16 TB S3 corpus, ALB) at **$2,058.78/month ($24,705.36 / 12 months)** — $2,015.78/month as shared, plus AWS WAF ($43.00/month: Web ACL on the ALB, AWS Managed Core/Known Bad Inputs/IP Reputation/SQL Database rule sets, Bot Control Common, rate-based rule, 20M requests/mo planning figure) added 23 Oct 2026 at the owner's direction — all in Asia Pacific (Mumbai) — is recorded in full in ARCHITECTURE §12.6 as a deployment alternative to this record's Fargate topology. The AWS team is asked to verify it (egress and corpus-size assumptions, Single-AZ vs Multi-AZ posture, Spot interruption handling, unpriced gaps) **whether or not it is adopted**; adoption remains a separate owner decision alongside this ADR. Estimate link: https://calculator.aws/#/estimate?id=50a088f3e134c1dcb06525fc8e6b32c956e71512 Updated estimate with WAF (23 Oct 2026): https://calculator.aws/#/estimate?id=85ab95370800b927cdd6e0264e90946f86481e2d — its WAF line shows $24.00/mo because the calculator cannot model Bot Control's subscription and per-request fees; §12.6 keeps the published-pricing $43.00/mo as canonical (owner decision, 23 Oct 2026).

Consequences (if accepted). (+) Managed, restart-proof, region-resident production with a cost profile dominated by one GPU node + Fargate tasks. (−) Estimated launch-tier run cost is materially above zero (Fargate + RDS Multi-AZ + GPU hours); OCR queue adds ingest latency by design (async).

Alternatives considered. EKS (rejected at launch scale: platform weight); single EC2 host with compose (kept as the small-deployment tier, ADR-0015 — not the production target); Lambda for the API (rejected: long-lived retrieval state, SSE streaming, and sidecar locality fit containers better); us-east-1 (rejected: residency, ADR-0017).

ADR-0017 — Data residency & compliance posture: India-first

Status: Accepted

Context. The corpus is Indian legal material, much of it confidential client data, sold to Indian firms. Residency and disclosure are product features, not legal footnotes.

Decision. Production data at rest lives in **ap-south-1** (ADR-0016); documents are encrypted in S3 (SSE-KMS) and the Private Vault is encrypted client-grade regardless of platform (ADR-0012). **Disclosed exception**: LLM/embedding provider calls may leave the region depending on the ADR-0008 outcome; the DPA/customer terms must name the provider and region story honestly, and the local-embeddings sidecar (ADR-0003) exists precisely to shrink that exception. Access is role-gated (ADR-0011); activity history provides the audit trail; private content is redacted from logs.

Consequences. (+) A residency story we can state in one sentence to a firm's IT committee. (−) Some best-in-class models are not available in-region; the provider decision (ADR-0008) must weigh that openly.

Alternatives considered. Global-by-default deployment (rejected); on-prem-only for enterprise (deferred — the Docker tier, ADR-0015, is the seed of that offer).

ADR-0018 — Adopt functionality, never identity, from reference products

Status: Accepted (standing product rule, owner-directed 20 Oct 2026)

Context. The product was researched against Harvey (design/IA) and Lexlegis (authenticated feature set), and evaluated against Dify (platform). The owner's rule ("Purple Cow"): clone information-architecture completeness, never identity.

Decision. REA-derived features are implemented **only after the owner's approval, with a reasoned recommendation**; naming, branding, and copy are always Juris Vault AI's own (the assistant persona is **Clause**; the matter workspace is a **Brief / Living Brief**; the competitor's assistant name is never used). Where a platform's license blocks adoption (Dify's modified Apache 2.0 forbids multi-tenant SaaS operation without a commercial license), we mine ideas and build natively — parent-child chunking (ADR-0009), annotated replies (Verified Answers), and the hit-testing panel all came from that evaluation with zero code reuse.

Consequences. (+) A defensible product identity and clean IP posture. (−) Every adopted idea costs a native build — accepted as the price of the moat (Indic OCR, cross-lingual retrieval, Living Brief memory, provenance receipts).

Alternatives considered. White-labelling or forking Dify (license- blocked); pixel-cloning competitor UX (identity risk, rejected by rule).

ADR-0019 — Fusion suite, rank-interleave leg merging, retrieval cache

Status: Accepted (23 Oct 2026; measured same day)

Context. A landscape evaluation of Spice.ai, Haystack, LlamaIndex, txtai, RAGFlow, Hyper-RAG and Verba converged on rank-fusion maturity as the transferable pattern (four of seven ship weighted/normalized fusion over plain RRF). Per ADR-0018 the patterns were rewritten natively in Rust — no sidecar runtime. The A/B programme then produced a more valuable finding than the tuning itself: per-stage retrieval on the 20-rule golden corpus is saturated (recall 1.000 at k=5 and k=10 in every configuration), while end-to-end failures traced to **Mix mode merging its vector/local/global legs by raw score across incommensurable scales** (cosines ~0.4–0.6 vs the graph legs' flat 0.85), which flooded the rerank pool with graph-neighborhood chunks and truncated the vector leg's best hits before generation.

Decision. (1) Legs merge by **rank interleave** (round-robin in each leg's own order, dedup by chunk id). (2) The vector↔BM25 fusion is a suite — RRF / weighted RRF / DBSF — with an **in-fusion statute term** (a query naming an Act lifts that Act's chunks ×1.35 by default, aliases matching abbreviated titles) and an optional recency-decay term. (3) A **retrieval-tier cache** with stale-while-revalidate, keyed by question + mode + scope + corpus fingerprint. (4) Production defaults stay conservative: RRF + statute term; weighted RRF and the post-fusion metadata boost are opt-in (measured stage-neutral to harmful — the identifier boost feeds the Act-blind BM25 leg).

Measured consequences. Golden set, test instance, 23 Oct 2026: 12/20 (pre-suite best) → 11/20 (suite, pre-merge-fix control) → **15/20** (merge fix in); all remaining failures are answer-wording only, with zero source-attribution misses.

Alternatives considered. Adopting Spice.ai as a sidecar (rejected — separate runtime, Alpha vector engines, ADR-0018); post-fusion metadata boosting as the statute mechanism (rejected by measurement — it cannot rescue a document the fusion cut already dropped).

ADR-0020 — Vector engines: Spice's lineup behind the VectorStore trait

Status: Accepted (23 Oct 2026; HNSW benched same day, S3 Vectors live test pending AWS credentials)

Context. The owner directed taking Spice.ai's vector engine as well. Research showed the "engine" is a configuration lineup, not an embeddable library: `s3_vectors` (AWS S3 Vectors, external), `duckdb` (in-process HNSW via the DuckDB VSS extension, M=16 / ef_construction=128 / ef_search=64 defaults), and `elasticsearch` (Enterprise-only — excluded on its face). SereneDB (ADR-0006) is the production store and is exact at the current corpus scale; the gap the engines fill is the scale-up path and the single-binary Docker tier, which today falls back to brute force.

Decision. Implement each Spice engine natively behind the existing `VectorStore` trait, selected by `LEGAL_RAG_VECTOR_STORE` (unset = legacy SERENEDB_URL behavior; production unchanged): (1) **`HnswVectorStore`** — pure-Rust HNSW with Spice's DuckDB-engine parameters, JSON-file persistence, unlink-on-delete, exact brute-force below 64 nodes. (2) **`S3VectorsStore`** — a hand-rolled SigV4 client for the S3 Vectors JSON API (no AWS SDK dependency), `doc_id` as filterable metadata for predicate pushdown, bucket/index auto-provisioned on connect; region default ap-south-1, where S3 Vectors is available (verified against AWS's regions table). (3) SereneDB remains the production default.

Measured consequences. 20,000 × 1024-dim bench (`crates/rag/examples/vector_bench.rs`): brute force 131 ms mean, recall@10 1.000 · SereneDB IVF 67 ms, 1.000 · HNSW ef_search=64 3.1 ms, 0.899 · ef_search=128 6.4 ms, 0.981. HNSW build at 20k is 186 s in this implementation (per-insert beam search) — a known cost to optimize before HNSW is recommended at large corpus scale.

Alternatives considered. Adopting the DuckDB crate + VSS extension (rejected: heavy C++ dependency, extension download at runtime, and Spice's own integration still lacks chunking/partition support); replacing SereneDB with HNSW as the default (rejected by measurement — SereneDB is exact and durable at today's scale).

Open decisions register (for the owner)

#	Decision	ADR	Needed from
1	Production LLM/embedding provider: Cloudflare Paid vs Bedrock vs self-hosted	0008	Jishan
2	Approve the AWS target architecture + launch-tier sizing	0016	Jishan
3	Payments for real money: YouPI-PG compliance review vs mainstream gateway	0013	Jishan
4	AWS-target vector store: SereneDB self-hosted vs RDS pgvector (production sandbox evaluation is DONE — SereneDB live since 23 Oct 2026; this is now only the AWS-target choice)	0006/0016	Jishan / AWS team
5	First Docker build + smoke test on a Docker host	0015	Infra team
6	Verify the shared 19 Oct 2026 AWS estimate ($2,058.78/mo incl. the WAF line added 23 Oct 2026, EC2-centric, 16 TB corpus) against ADR-0016 — see ARCHITECTURE §12.6	0016	AWS team

Register discipline: when a decision lands, the corresponding ADR's status changes in the same commit, the change is logged in CHANGELOG.md, and this register is updated.

Document control: ADR set v1.2, authored 21 Oct 2026 by the Juris Vault AI Team (owner: Jishan). New decisions get new records; accepted records are never edited except to mark them Superseded.